Revisiting richACL in the AI Era

Scenario: photo processing and sharing with AI agents

Coly Li <colyli@fnnas.com>

fnOS kernel team

Dec 11, 2025

fnOS and fnNAS

fnOS is an AI-native NAS operating system for home and small businesses.

Key features:

- No hardware lock-in, users can freely choose hardware to install fnOS (now x86-64, aarch64 in future)
- User space applications + AI agents + Debian distribution.
- fnNAS = fnOS + hardware
- More than storage, local deployed AI modules and agents find further interconnection from the photos (movies and music as well).
- AI processed photos are shared within fnNAS device among users, or over internet with external users.
- Typically users in fnNAS are family members or small business.

Photos and AI agents

Users upload their photos from mobile phone, laptop or desktop up to their private directories on fnNAS device.

Different AI agents work to:

- Analyze photos (content, location, face, ...) in private directories.
  [access permission: read, write, non-deletable]
- Generate interconnections among the photos, create hard-links of the photos in different sub-directories (a.k.a categories) for the detected relationships, recommend the categories to photos owner.
  [access permission: read, write, non-deletable]
- If users share the recommended photo categories with other users in same fnNAS device, create identical categories sub-directories with hard-links in public directories.
  [access permission: read, write, non-deletable]

Photos and AI agents (Cont.)

- Analyze all shared photos in public directories, find out more interconnections, generate more photo categories and recommend them back to users.
  [access permission: read, write, non-deletable]
- If users share the recommended photo categories with other users in same fnNAS device, create identical categories sub-directories with hard-links in public directories.
  [access permission: read, write, non-deletable]
- Delete specific photo(s) if some user doesn't want himself/herself to appear in the recommended photos, requires an AI agent to identify his/her face and delete the photos from shared categories.
  [access permission: read, delete, non-writable]

Positive emotion values: e.g. A category with photos from mobile phones of father and daughter for a family excursion years ago, they can experience different view of each other, recall the sweet memory and realize how time flies.

Three Types of AI Agents

1. **Agents on private photos**:
   - Reads user's private photos.
   - Output new recommended photo categories only to owner.
   - Can be found and installed from AI agent store.
2. **Agents on public photos**:
   - Analyzes all shared photos.
   - Generates cross-user photo categories, recommend the result back to users.
   - Can be found and installed from AI agent store.
3. **Privacy Agent**:
   - Trusted agent provided by fnOS.
   - Delete face-matched photos from public shared categories, if required by face owner.

The **AI agent store** will provide multiple AI agents with diverse features and capabilities, most of them are developed by third-party developers.

Critical security principle:
third-party agents must never delete/corrupt private or public shared photos.

AI Scenarios Need More Powerful Access Controls

1, Explicit DENY

- User father shares several photo categories to family, but wants to **explicitly deny** user son.
- User father wants to **explicitly deny** all family members from a photo category except for user mother.
- POSIX ACL has no explicit deny.

2, Writable while Non-Deletable

- Agents on private photos: create hardlinks, read/write metadata, **non delete**
- Agents on public photos: create hardlinks, read/writemetadata, **non delete**
- Agent to handle privacy requests: read + **delete hardlinks**, no write

AI Scenarios Need More Powerful Access Controls (Cont.)

3, Efficient Recursive Access Control Inherintance

- third-party AI agent may have its own user name pre-defined by the developer.
- When user or sys admin wants to install a third-party AI agent,
  - Access control for a new user might be set.
  - Access control rules are recursively applied on all sub-directories and photos.
- When user or sys admin wants to uninstall a third-party AI agent,
  - Reverse operations will be performed.
- Performance matters,
  - AI agent installation/deletion is frequent operations.
  - In-kernel access control inheritance is much faster and more efficient.

AI Scenarios Need More Powerful Access Controls (Cont.)

4, Path-Dependent Access Controls

- Identical file is accessed via different path,
  - From different photo categories by different AI agent.
  - From different photo categories shared to different users.
- Granting path-dependent access controls makes multiple AI agents scenario possible and more flexible.

Notice:

- The first 3 requirements are **subset of richACL capabilities**.
- The 4th requirement is not in richACL, but is extremely important and mandatory.

Current Industry Workarounds

1. **eBPF hooks and rules**
   - Complicated to manage/persist.
   - Performance: $O(N_{\text{files}} \times N_{\text{rules}})$ in worst case.
   - Must set DAC (rwx bits) to 777, confuses users and applications. - ACLs inheritance is very hard to implement efficiently.
2. **LSM hooks and rules**
   - Similar challenges as eBPF hooks.
3. **Custom VFS patches**
   - Incomplete, non open source, vendor-locked, no standard format.

Proposal: Reconsider richACL for Mainline Kernel

richACL is desired to be upstreamed.

- richACL minimal subset:
  - **explicit DENY**
  - **writable while non-deletable - recursive ACLs inheritance**.
- **path-dependent access control**
  - Need generic_permission() and inode_permission() has a 'path' parameter to the inode, and check access permission with 'path' too.

Industry trends: It is industrial agreement that NAS is the only device to deploy AI in home and small business scenario. More powerful access controls are desired in AI era, and richACL is the most promising implementation in front of us.

Thank You!

Questions?